Guía de Trabajos Prácticos

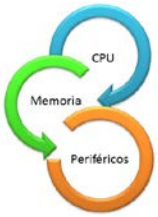
UNIDAD TEMÁTICA 2

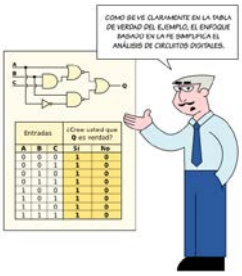
SISTEMAS COMBINACIONALES

ISBN 978-950-42-0158-8

9 789504 201588

Versión 2.0 – Agosto 2018





EJERCICIOS A RESOLVER EN CLASE

EJERCICIO 1:

Diseñe un sistema combinacional que accione el motor de un limpiaparabrisas sabiendo que éste funciona cuando la llave general del auto está en contacto, y además, se activa el control del limpiaparabrisas, pero si desactiva este último, el motor sigue funcionando hasta que las escobillas lleguen al costado izquierdo.
Obtener la solución más simple, e implementarla con las compuertas correspondientes.

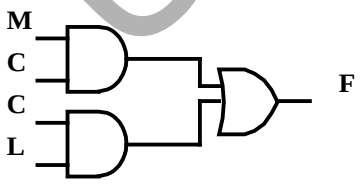
RESOLUCIÓN:

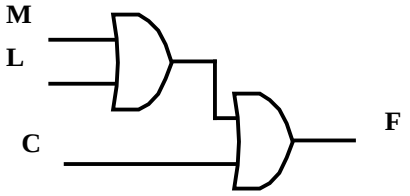
Vamos a llamar **C** a la variable de la llave de contacto del auto.
Vamos a llamar **M** a la variable que acciona el motor del limpiaparabrisas.
Vamos a llamar **L** a la posición de la escobilla, vale 1 cuando está a la derecha.

C	M	L	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

C \ ML	00	01	11	10
0	0	0	0	0
1	0	1	1	1

$$F = MC + LC = C(M + L)$$

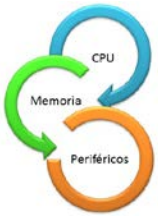


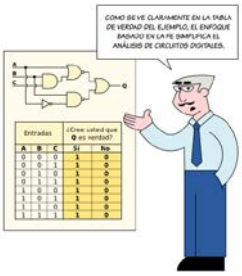


EJERCICIO 2:

Realizar el diseño e implementación de dos llaves de luz colocadas al pie y cima de una escalera; de tal manera que pueda prender la luz de la llave de abajo y apagarla con la llave de arriba y viceversa.

UTN Mendoza





EJERCICIO 3:

Utilizar decodificadores para implementar las siguientes funciones:

1. $f(a,b,c,d)=\sum_{4}(0,1,7,8,10)$
2. $f(a,b,c,d)=\sum_{4}(1,2,3,5,,8,12)$

EJERCICIO 4:

Realizar el diseño de un comparador de dos números de dos bits c/u. Este sistema tiene que tener 3 salidas la de mayor, menor e igual. Realizar el diseño con compuertas Nand.

EJERCICIOS A RESOLVER POR EL ALUMNO

EJERCICIO 1:

Diseñar un sistema combinacional que ante la excitación de 4 variables de entrada genere salidas que activen un display de 7 segmentos para la representación de los dígitos decimales sobre el mismo. Considerar que las combinaciones de entrada corresponden al código BCD 8421, y que las restantes a las 16 posibles a las letras del alfabeto a, b, c, d, e y f. Obtener la solución más simple, e implementarla con las compuertas NOR.

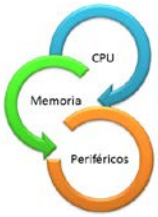
EJERCICIO 2:

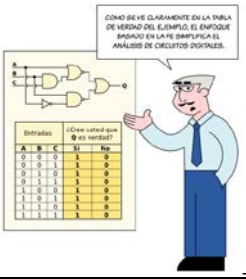
Diseñe un sistema digital de semaforización para una estación terminal de ferrocarril. Como se observa en la figura, a dicha estación acceden 4 líneas de diferentes tipos de servicios con las siguientes prioridades:

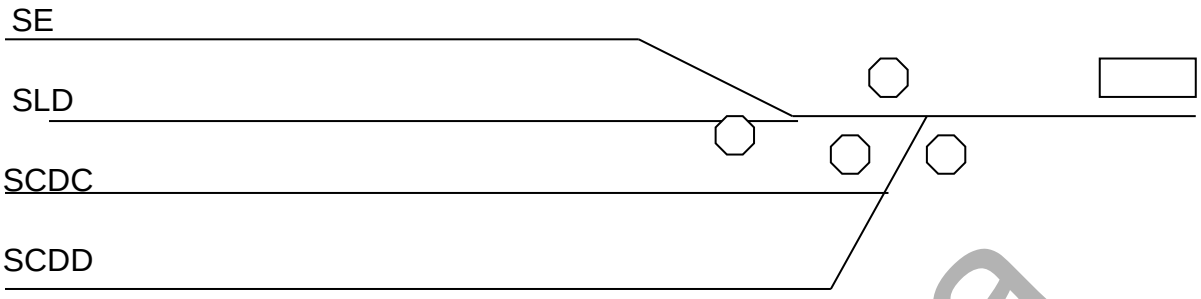
1) **S**ervicio **E**xpreso (SE)
2) **S**ervicio **C**orta **D**istancia **D**iferencial (SCDD)
3) **S**ervicio **C**orta **D**istancia **C**omún (SCDC)
4) **S**ervicio **L**arga **D**istancia. (SLD)

Sensores en cada línea indicarán ó no la presencia de vehículos, y semáforos en rojo ó en verde detendrán ó habilitarán el paso respectivamente. Obtener la solución más simple, e implementarla con las compuertas correspondientes.

UTN Mendoza





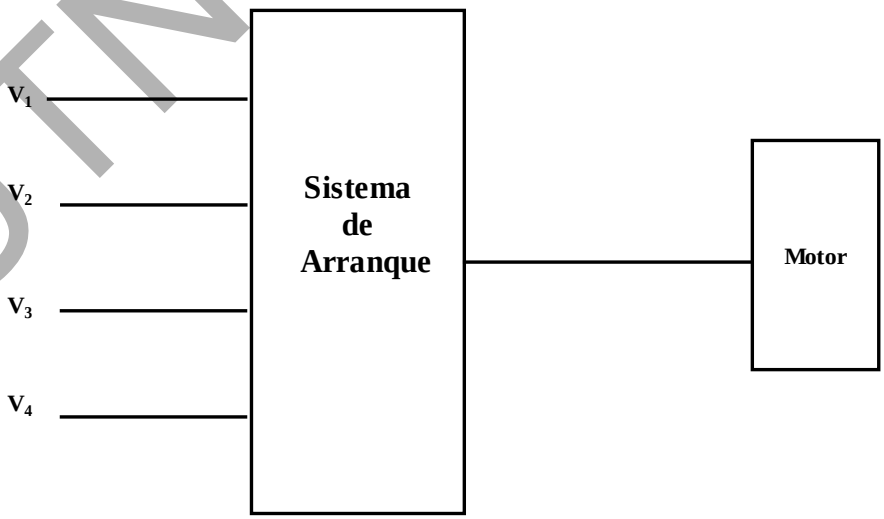


EJERCICIO 3:

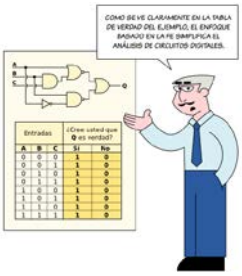
Un registro de salida en paralelo A_0 a A_3 contiene un carácter codificado en BCD 8421. Realizar un generador de paridad para agregar el correspondiente bit A_p que permita elegir paridad par o impar, con una llave selectora. Utilizar decodificadores y multiplexores

EJERCICIO 4:

El diagrama de la figura indica el sistema de arranque de un motor eléctrico. Existen cuatro variables binarias de control las cuales permitirán el arranque del motor sólo cuando existe paridad impar entre ellas. Hay dos casos especiales, llamados de emergencia que permitirán arranque, y ocurren cuando todas las variables están en uno ó cero simultáneamente.
Realizar con compuertas.



UTN Mendoza



EJERCICIO Nº 5

Especificaciones

- ***Objetivo***: Diseñar un circuito digital, capaz gobernar un robot, haciendo que éste siga una línea negra pintada sobre un fondo blanco.
- ***Sensores***: El robot está dotado de dos sensores digitales capaces de diferenciar el color negro del blanco. La salida de estos sensores es '0' cuando leen blanco y '1' cuando leen negro. Denominaremos a este bit como C:

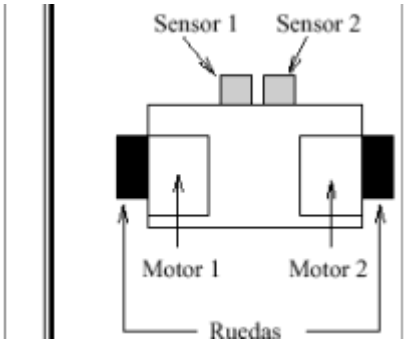
Sensor	C
Color Blanco	0
Color Negro	1

- ***Motores***: Dos motores de corriente continua que son controlados cada uno mediante dos bits, denominados S y P, descritos mediante la siguiente tabla de verdad:

P	S	Motor
0	0	Parado
0	1	Parado
1	0	Giro derecha
1	1	Giro izquierda

El bit **P** es el bit de 'Power'. Indica si el motor está conectado o no. El bit **S** es el del sentido de giro. Según su valor el motor girará a la derecha o a la izquierda (siempre que el motor esté activado, con P=1).

- ***El robot:*** El esquema del robot es el siguiente (visto desde arriba):



UTN Mendoza

- ***Algoritmo:*** El algoritmo para seguir la línea negra es muy sencillo. Mientras los dos censores detecten negro, el robot deberá avanzar. Cuando el censor de la derecha detecte blanco y el de la izquierda negro, el robot girará a la izquierda y cuando ocurra el caso contrario girará a la derecha. Si ambos censores leen blanco permanecerá parado. Esto se esquematiza en la siguiente figura:

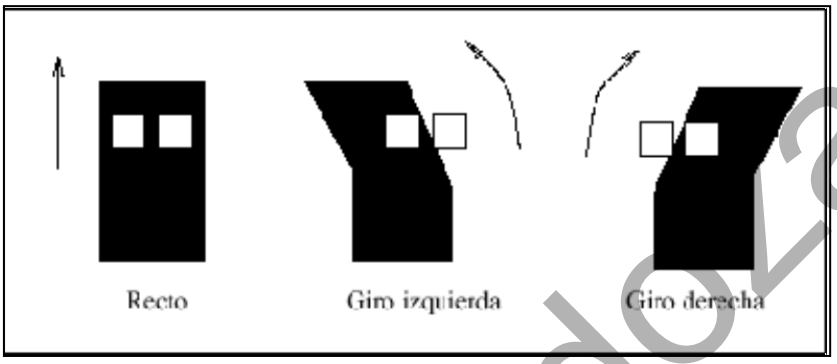


Diagrama de bloques

Como primera fase del diseño tenemos que entender qué es lo que se nos está pidiendo y determinar el aspecto que tiene el circuito que hay que realizar. El circuito tendrá dos entradas provenientes de los censores, C1 y C2, y cuatro salidas, dos para cada motor: S1, P1 S2 y P2.

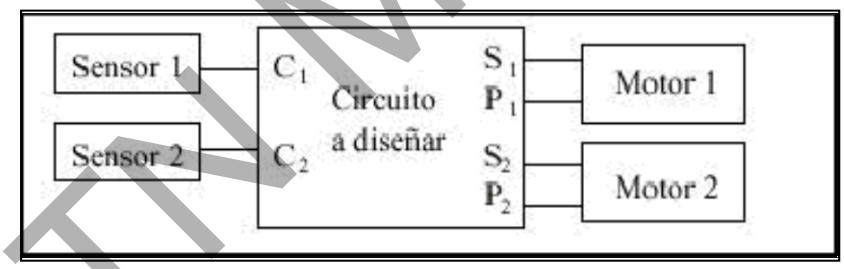


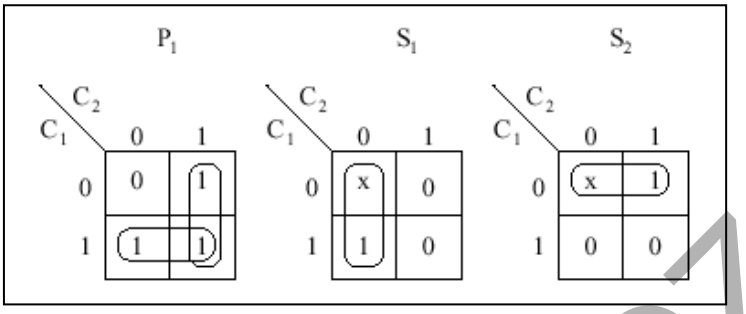
Tabla de verdad

Ahora hay que definir el comportamiento del circuito, utilizando una tabla de verdad. Este comportamiento nos lo da el algoritmo de seguir la línea. La tabla de verdad es la siguiente:

C_1	C_2	S_1	P_1	S_2	P_2
0	0	x	0	x	0
0	1	0	1	1	1
1	0	1	1	0	1
1	1	0	1	0	1

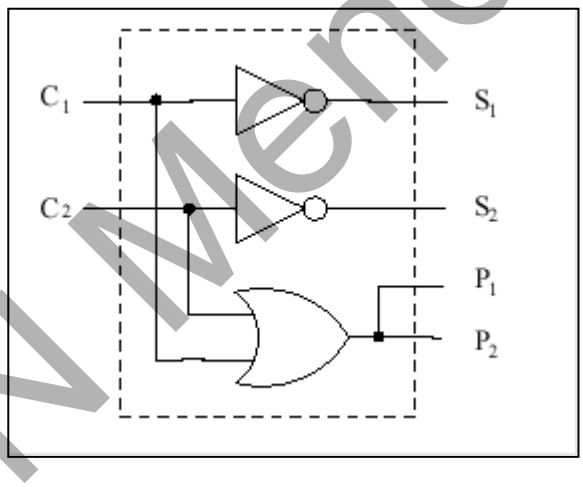
UTN Mendoza

Con una ’x’ se han marcado las casillas de la tabla de verdad que es indiferente su valor. Según nos convenga puede valer ’0’ ó ’1’. El Karnaugh del circuito es:

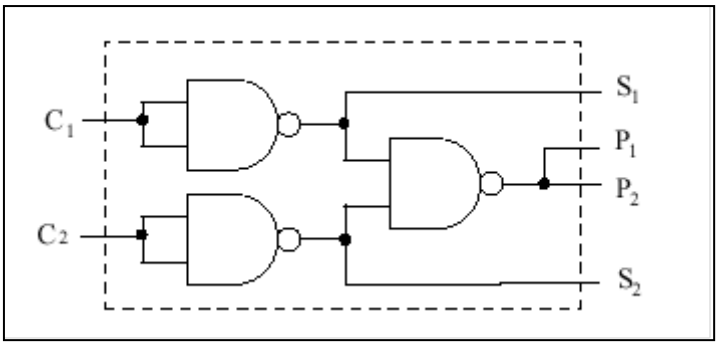


Implementación del circuito

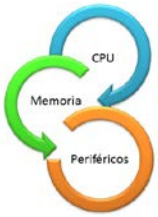
El circuito, implementado con puertas lógicas básicas es el siguiente:

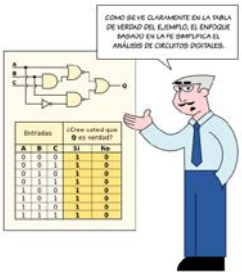


Si lo construimos utilizando puertas TTL, necesitamos dos integrados, uno para los inversores y otro para la puerta OR. Si en vez de ello lo implementamos sólo con puertas NAND, el circuito es el siguiente:



UTN Mendoza

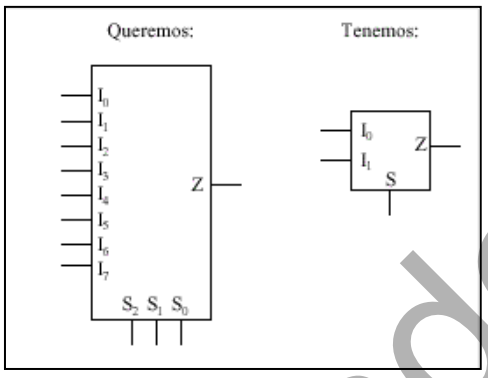




Ejercicio Nº 6

Construir un multiplexor de 8 canales a partir de multiplexores de 2 canales.

La solución es conectarlos en cascada:



Primero colocamos una columna de 4 multiplexores de dos entradas, para tener en total 8 entradas. Todas las entradas de selección de esta primera columna se unen. Por comodidad en el dibujo, esto se representa mediante una línea vertical que une la salida S de un multiplexor con el de abajo.

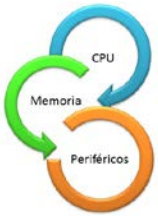
A continuación colocamos una segunda columna de 2 multiplexores de 2 entradas, también con sus entradas de selección unidas. Finalmente colocamos una última columna con un único multiplexor de 2 entradas.

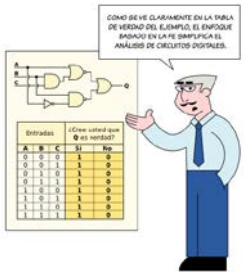
Colocados de esta manera, conseguimos tener un multiplexor de 8 entradas y tres entradas de selección. La única consideración que hay que tener en cuenta es que la entrada de selección de los multiplexores de la primera columna tiene peso 0, la segunda peso 1 y la última peso 2:

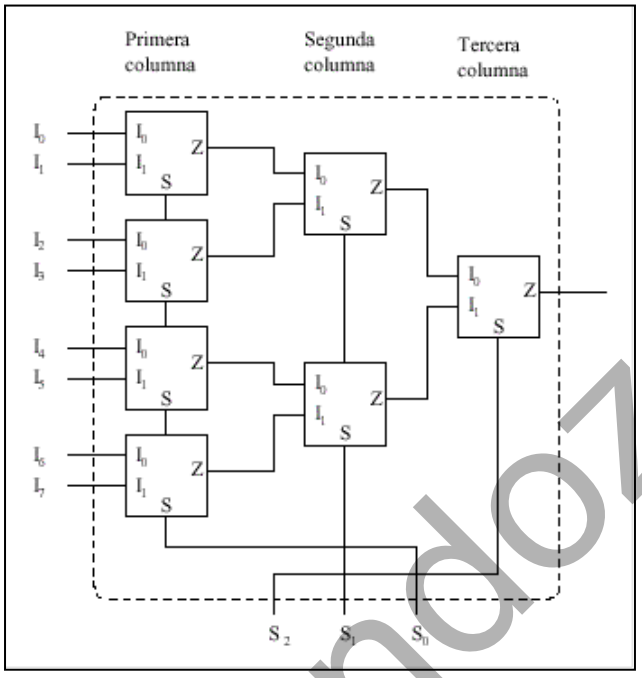
Vamos a comprobar qué ocurre si seleccionamos el canal 6. Introducimos en binario el número 6 por las entradas de selección S2, S1, S0.

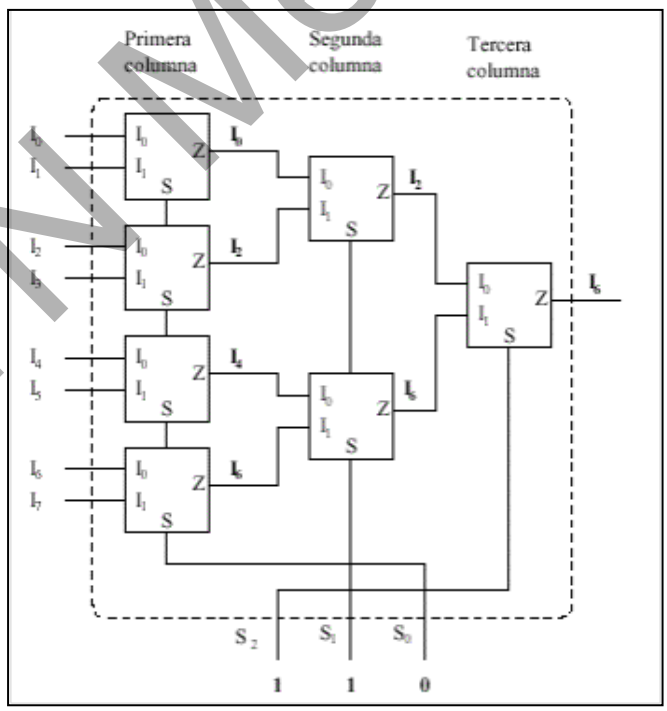
Por la entrada S de los multiplexores de la primera columna se introduce un ’0’, por lo que estos multiplexores sacan por sus salidas lo que hay en sus entradas I0. Por la entrada de selección de los multiplexores de la segunda columna se introduce un ’1’ por lo que están seleccionando su canal I1. A la salida de estos multiplexores se tendrá I2 e I6. Finalmente, el multiplexor de la última columna está seleccionando su entrada I1, por lo que la salida final es I6.

UTN Mendoza

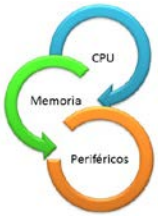

CPU
Memoria
Periféricos

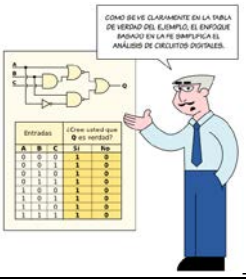



Primera columna
Segunda columna
Tercera columna
I_0 I_1 I_2 I_3 I_4 I_5 I_6 I_7
Z
S
S_2 S_1 S_0


Primera columna
Segunda columna
Tercera columna
I_0 I_1 I_2 I_3 I_4 I_5 I_6 I_7
Z
S
S_2 S_1 S_0
1 1 0

UTN Mendoza





Ejercicio Nº 7

Realizar un detector de desbordamiento aritmético. El intervalo de valores que podemos representar mediante números de n bits en el sistema numérico de complemento a dos es:

$$-2^{n-1} <= N <= 2^{n-1} - 1$$

Cualquier operación aritmética que intente generar un valor fuera de este intervalo producirá una condición de desbordamiento. En estos casos el número resultante de n bits no es una representación válida del resultado; por tanto hay que detectar todas las condiciones de desbordamiento, de modo que no se utilicen estos resultados nos válidos.

En complemento a dos, se tienen condiciones de desbordamiento cuando se suman dos valores positivos cuya suma es mayor que $2^{n-1} – 1$ o dos valores negativos cuya suma es menor que $-2^{n-1} – 1$. En ambos casos el resultado tiene el bit de signo incorrecto. Por lo tanto, detectamos el desbordamiento observando los bits de signo de los operandos y del resultado.

En la siguiente tabla se observan las situaciones que pueden ocurrir en la etapa más significativa de un sumador de n bits. Los bits a_{n-1} y b_{n-1} representan los bits de signo de los números a sumar, así que son las entradas de esta etapa, junto con el bit de acarreo c_{n-2}. Las salidas de esta etapa son C_{n-1} y S_{n-1}.

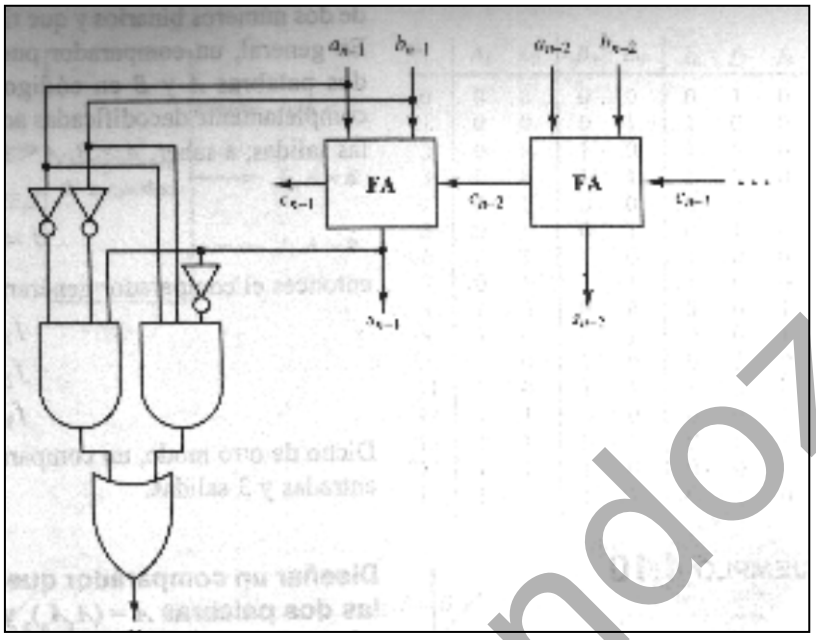
Ocurre desbordamiento en dos casos: la suma de dos valores positivos que producen un bit de signo S_{n-1}=1, lo que indica resultado negativo, y la suma de dos números negativos que produce un bit de signo S_{n-1}=0, lo que indica un resultado positivo. La expresión lógica de desbordamiento V es:

$$V = \overline{a_{n-1}}.\ \overline{b_{n-1}}.\ S_{n-1}. + a_{n-1}.\ b_{n-1}.\ \overline{S_{n-1}}$$

	Salidas del sumador		Desbordamiento
c_{n-2}	c_{n-1}	S_{n-1}	V
0	0	0	0
1	0	1	1
0	0	1	0
1	1	0	0
0	0	1	0
1	1	0	0
0	1	0	1
1	1	1	0

La siguiente figura muestra el circuito de la ecuación anterior.

UTN Mendoza



UTN Mendoza